

Claude Basics

What Claude is & model tiers	Capabilities, limitations & prompting
Claude web interface — one-liners & demos	Claude Code — slash commands & context management

01 What Claude Is

Claude is Anthropic's family of AI models — a conversational assistant that reads, reasons about, and generates text (and can also work with images, code, and files), built to be helpful, honest, and safe by design rather than by afterthought.

How people actually reach it

Claude isn't one single product — it's the same underlying model surfaced through different tools depending on the job:

- **Claude.ai / the Claude app** — the everyday chat interface, web, desktop, and mobile.
- **Claude Code** — an agentic coding tool that works from the terminal, IDE, or desktop app, covered in Session 2 today.
- **Claude Cowork** — an agentic tool for broader knowledge work: research, analysis, multi-step tasks with files.
- **The Claude API / Console** — for teams building Claude into their own products (Day 3).
- **Claude for Chrome / Excel / PowerPoint** — task-specific agents inside tools people already use.

02 Model Tiers — When to Use Which

Claude comes in a few weight classes. The trade-off is always the same one: speed and cost on one side, depth of reasoning on the other. Picking the right tier for a task matters as much as the prompt itself.

TIER	BEST FOR	TRADE-OFF
Haiku	High-volume, low-latency work: quick classification, simple extraction, chat where responsiveness matters more than depth.	Fastest and cheapest — but shallower reasoning on genuinely hard, multi-step problems.
Sonnet	The default for most day-to-day work — writing, coding, analysis, and agentic tasks like Claude Code.	The balance point: strong reasoning at a cost and speed that holds up for everyday use.
Opus	The hardest problems — deep multi-step reasoning, complex codebases, high-stakes analysis where quality outweighs speed.	Slower and more expensive; reserve it for tasks that actually need the extra depth.
Fable	Newest tier in the Claude 5 lineup, sitting alongside Haiku, Sonnet, and Opus.	Positioning and best-fit use cases are still emerging — confirm current guidance before recommending it as a default.

03 Realistic Capabilities vs. Limitations

Set expectations here honestly. The teams that get the most out of Claude are the ones who know precisely where to lean on it and where to stay in the loop.

WHAT IT'S GENUINELY GOOD AT

- Drafting, editing, and restructuring writing at speed
- Reading and reasoning over long documents or codebases
- Multi-step tasks when broken into clear instructions
- Coding: writing, debugging, explaining, refactoring
- Using tools it's given — search, files, other systems — to fill gaps in its own knowledge

WHERE TO STAY IN THE LOOP

- Claude's training data has a cutoff — it doesn't know what happened after, unless it searches
- It can state incorrect things confidently ("hallucination") — verify anything load-bearing
- No memory across sessions by default — context has to be re-supplied or persisted deliberately (CLAUDE.md is one way, covered next session)
- Output quality tracks input clarity — vague asks get vague answers
- It's not a source of truth on its own for numbers, citations, or current events without a way to check them

04 Prompting Fundamentals

Three steps account for most of the gap between a frustrating first response and a genuinely useful one.

01 Set the Stage

Share your role, objectives, and relevant context with Claude.

02 Define the Task

State clearly what action you want: write, analyze, build, research.

03 Specify the Rules

Indicate desired style, tone, length, and format.

Example Prompt

EXAMPLE PROMPT

"I'm the marketing lead at an indie streaming startup. Research the streaming market and identify key trends, competitor positioning, and growth opportunities. Structure it as a 5-page professional report with an executive summary, market analysis, and competitive landscape."

05 Claude Web Interface — Features & Demos

Beyond a plain chat box, claude.ai bundles a set of features that shape how people actually get from a blank prompt to finished work. Each one below is a mode, a workspace, or an integration worth showing live.

Use Cases

Different "modes" or starting points tailored to common jobs (writing, coding, analysis, etc.) so you don't start from a blank prompt.

USE CASE A new user wants a quick way to draft a marketing email without knowing how to prompt well.

DEMO Open claude.ai → pick a use-case template (e.g., "Write") → show how it pre-fills a structured prompt.

Projects

A workspace that groups chats, files, and custom instructions around a specific goal, so context persists across conversations.

USE CASE A team building a product spec keeps all research docs and chats in one Project instead of re-uploading files every time.

DEMO Create a Project → upload 2–3 reference files → start a new chat inside it and ask a question that requires those files, showing Claude already "knows" them.

Artifacts

A side panel that renders and lets you iterate on generated content (code, docs, diagrams, mini web apps) outside the chat flow.

USE CASE Generating a working React component or an HTML landing page you can preview and tweak live.

DEMO Ask Claude to "build a simple to-do list app" → show the Artifact panel rendering and updating as you ask for changes.

Connectors

Integrations that let Claude read/act on your external tools (Slack, Jira, GitHub, etc.) directly inside a conversation.

USE CASE Ask Claude to summarize unread Slack messages or catch up on a channel without leaving the chat.

DEMO Connect Slack → ask "summarize what I missed in #eng-updates today."

Plugins

Extend Claude Code (or Claude) with reusable, shareable capabilities — think of them as installable add-ons for custom workflows.

USE CASE A team distributes a shared set of custom slash commands or tools across their whole engineering org.

DEMO (best shown in Claude Code) install a plugin and run a custom command it adds.

Skills

Packaged, reusable instructions/best-practices Claude reads before doing a task (e.g., how to build a proper .docx or .pptx).

USE CASE Ensuring every generated Word doc or slide deck follows the same formatting standards automatically.

DEMO Ask Claude to "create a Word doc report" and show it consulting the docx Skill before generating the file.

Research

An agentic mode where Claude autonomously runs many searches across sources and compiles a sourced, in-depth report.

USE CASE Competitive market analysis across a dozen companies that would take hours to research manually.

DEMO Ask a broad question ("compare the top 5 CRM tools for SMBs") → trigger Research → show the multi-source report it returns.

Web Search

Lets Claude pull live, current information from the internet instead of relying only on training data.

USE CASE Getting today's stock price, the latest news, or checking if a fact has changed since Claude's knowledge cutoff.

DEMO Ask "who won the game last night" or "what's the latest version of X" and show Claude searching before answering.

06 Claude Code

A first look at Claude Code before Session 2 goes deep — how you talk to the session itself, and how it manages what it remembers as a project grows.

Slash Commands

What they are

Typed commands starting with `/` that control the Claude Code session itself (not the codebase) — think of them as session-management shortcuts.

Core commands

COMMAND	ONE-LINER	USE CASE
/help	Lists all available commands and what they do	Forgot a command name mid-session
/clear	Wipes the current conversation and starts fresh	Switching to an unrelated task so old context doesn't confuse Claude
/compact	Summarizes and compresses older conversation history to free up context space	Long session approaching context limits, but you want to keep working
/exit	Ends the Claude Code session	Done working for the day
/init	Scans your project and generates a <code>CLAUDE.md</code> with project context/rules	Onboarding Claude Code to a new repo for the first time

DEMO

In a real repo — run `/init` first (show the generated `CLAUDE.md`), do some work, run `/compact` once context builds up, then `/clear` to reset before a new unrelated task.

Permission Modes

What they are

Auto, Plan, and Manual aren't slash commands — they're permission modes that control how much confirmation Claude Code asks for before it takes a **tool action**, like editing a file or running a shell command.

AUTO Claude executes tool actions (file edits, commands) on its own with minimal confirmation — fastest, best for well-defined or low-risk tasks.

PLAN Claude proposes a plan first and waits for your approval before it runs any tool actions — best for complex or risky changes.

MANUAL Claude asks for approval before each individual tool action — most control, most oversight.

Understanding Context Management with Claude Code

Why it matters

Claude Code has a limited context window per session, so managing what stays "in memory" (via `/compact`, `/clear`, and `CLAUDE.md`) directly affects speed, cost, and accuracy.

USE CASE

A multi-hour coding session where irrelevant early back-and-forth is bloating the context and slowing/cluttering responses.

DEMO

Show `/context` (or `/cost`) to check usage, then `/compact` with a focus instruction (e.g., "keep the auth flow and test results") to show selective retention.

07 Suggested Demo Flow (End-to-End)

One continuous walkthrough that strings the web interface and Claude Code together, rather than showing each feature in isolation.

01 Web

Create a Project → upload a doc → connect Google Drive → ask Claude to research a topic and summarize using the connected file.

02 Web

Ask Claude to build a small app → show it open in Artifacts → iterate on it live.

03 Code

Open a repo → `/init` → make a change in Plan mode → `/compact` → `/clear` → `/exit`.

Key takeaways from Session 1

- Claude is one model family, reached through different tools depending on the job — chat, Claude Code, Cowork, or the API.
- Pick the model tier for the task: default to the mid-tier, drop down for volume, step up for genuinely hard problems.
- Know where Claude is strong (drafting, reasoning over content, coding) and where to verify (facts, recency, anything load-bearing).
- Set the stage, define the task, and specify the rules — the three steps that separate a mediocre response from a great one.
- The web interface is more than chat — Projects, Artifacts, Connectors, and Skills each solve a different "starting from a blank prompt" problem.
- In Claude Code, `/init`, `/compact`, and `/clear` are the three commands that keep a long session fast and accurate.

Important Links

Official Anthropic documentation for the three areas covered in this session.

TOPIC	OFFICIAL DOCUMENTATION
Web UI	claude.ai
Use Cases	Claude use cases
Slash Commands	Claude Code commands reference